

Report on Machine Learning and Deep Learning Approach

By

Arunkumar Venkatesan

Jiteshkumar Vasavada

Naveen Kumar Maddu

Gruden Daniel

Ansoldi Marco

1 Input Parameters

Two separate casters, designated **TSC1** and **TSC2**, were considered for the analysis. The steel grades relevant to this study fall into the following categories:

Low Carbon: SAE1006

Medium Carbon: SS400 and S275JR

Model Levels: Heat-level, Slab-level

Time Period for Data Modeling: 01/08/2023 to 14/07/2024

Model Prediction: positive prediction of the model means the coils which are predicted as rejected coils.

Note: Casting process is performed in sequences which last for approximately for 15 to 20 hours. Each sequence consists of 20 to 30 heats. Each heat consists of 5 to 6 slabs.

1.1 Number of Features

The raw dataset consists of **248 raw sensor parameters**. Most of the parameters are having minimum and maximum values defined by the Danieli Automation. These values were used to check the quality of the sensor values and then features are computed. If the sensor values are outside the ranges, then those samples were not considered for feature calculation. Based on the parameters, additional features were also computed using sensor data (resample to 1 second), where statistics used are maximum, minimum, and standard deviation for each slab. For features calculation, we considered:

- Statistics: standard deviation, mean, range of variation, slope
- Rolling Window: 30s, 120s

Additional to above statistical features we also have computed few more feature based on few technical observations:

- Local difference and Local Time Features for Temperature, Stopper rod and Tundish weight sensor parameters

- Local mean temperature of the region for temperature sensor parameters (as shown in Figure 1).

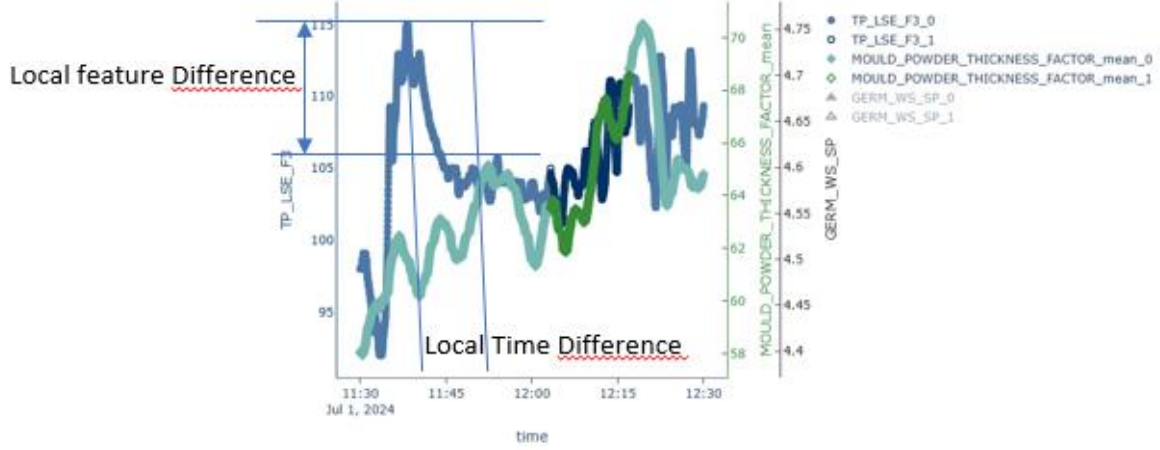


Figure 1 - Feature Engineering Example

After feature engineering, considering all those parameters and additional features that are developed based on technical observations, the total number of features used for the slab-level model were around **2938 features**.

1.2 Target Variable

The target variable differs across models. For the **slab-level model**, the target variable is derived from customer feedback through manual classification, indicating whether a slab is rejected or accepted. At the plant site, operator observes the coil in Surface Inspection System supplied by IMS company and he/she decides based on the IMS output and manual judgement whether the coil is to be rejected or not. IMS system provides the percentage length of the defect

$$\text{percentage length of the defect} = \frac{\text{total length of the defects}}{\text{length of the coil}} \times 100$$

Customer criteria for classifying the coils into different quality is mentioned in the Table 1. Based on these criteria, coils are considered defective, if continuous length of the defect is more than 2% or discontinuous length of the defect is more than 5%. For **heat-level model**, heat is considered

defective, if the at least 1 slab is rejected (based on manual observation) in the heat. Each heat produces around 5 to 6 slabs.

Table 1 Customer criteria to classify the coil in primary, acceptable and secondary quality from output (defect length and image with overlay of the defects) of the IMS system

Criteria	Primary	Acceptable	Secondary
continuous crack length	$\leq 2\%$	$> 2\%$ and $\leq 3\%$	$> 3\%$
discontinuous crack length	$\leq 5\%$	$> 5\%$ and $\leq 7\%$	$> 7\%$

1.3 Data Preprocessing

From the raw data, initial level of filtering is done based on the process understanding as mentioned below,

1. Casting speed > 4 m/min

- The slabs having casting speed more than 4 m/min are only considered, since the casting process attains stationary condition.

2. Slab Thickness Filter:

- Several batches of slabs are produced with different thickness profiles (72 mm, 76 mm, 82 mm, 86 mm etc.). Only three thicknesses (72 mm, 76 mm, 82 mm) were selected which contributed more than 98% of the production.

3. Filtering initial 2 slabs and the last slab of a sequence:

- In continuous casting, sequence is uninterrupted (continuous) process of casting contains multiple heats (approx. 15 to 30 heats).
- Based on the process understanding, two slabs at the start of the sequence and last slab of the sequence are filtered from the data because defects in those slabs may be due to start of the process or end of the process and root cause may not lie in the mold.

4. Removing time periods having no production

- Sensor data always record values irrespective of machine status (running or idle) however based on certain parameters sequence start and sequence end time stamp are identified, and this period is considered as production period when the casting is in process.

The data related preprocessing steps are described below:

1. NaN Analysis

Features were evaluated for the proportion of missing values on a month-wise basis. Any feature with **more than 5% NaN values** was removed, indicating either sensor malfunction or missing data. The timeline for this analysis was selected in such a way that there is no disturbance or operational shutdown that would disrupt the parameters. One such example for NaN values variation and removal is mentioned in Figure 2.

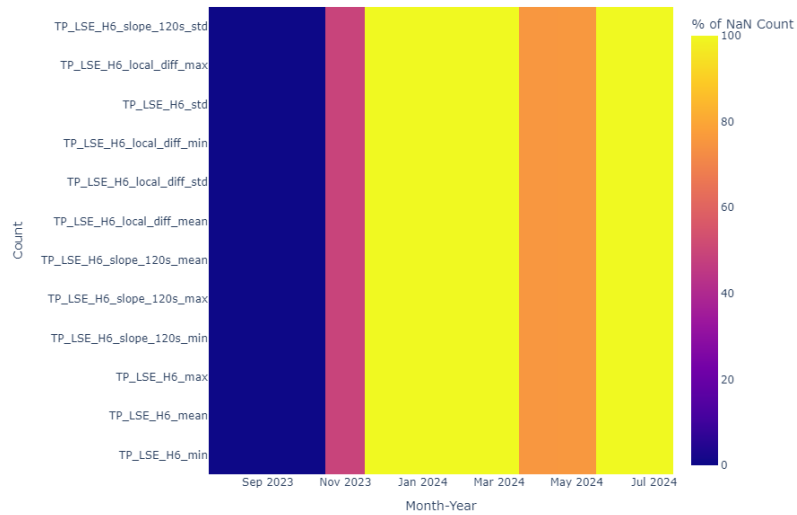


Figure 2 Example of Month-wise NaN value data

2. Outlier Detection

Outliers are detected by computing quantiles at fine intervals of 0.01 within the ranges 0.9–1.0 and 0–0.1. In the upper range (0.9 to 1.0), an outlier is flagged when the value at quantile $i+1$ shows a sharp increase (more than 50%) compared to quantile i . After identifying the features having outlier, these outlier values of respective features are manually validated and used to filter the data samples.

- For each step = 0.01 in quantile ranges of $[0.9, 1]$, $\frac{q_{i+1}-q_i}{q_i} > 0.5$, then the outlier value of the feature is calculated for the quantile q_{i+1} .
- For each step = 0.01 in quantile ranges of $[0,0.1]$, $\frac{q_{i+1}-q_i}{q_i} > 0.5$, then the outlier value of the feature is calculated for the quantile q_i .

After determining the lower and upper outlier thresholds based on these quantile comparisons, the dataset is filtered accordingly for subsequent analysis.

Data Drift Analysis

- Using the *Evidently AI* library, drift was computed between:
 - Train vs. Test datasets
 - Train vs. Validation datasets
- Features exhibiting significant drift were removed to ensure model generalization.

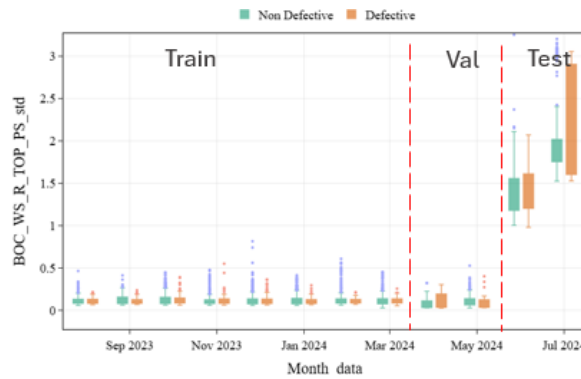


Figure 3 - Example of a removed feature that displays significant value drift across Train-Test.

After all filtering steps, the final features set consisted of **1913 features**. A detailed tabular column consisting of the steps involved is displayed below.

Actions Performed	Number of samples present	Number of Features present
Data Extraction for Caster 2	118060	3925
Selection of Low Carbon Slabs	74778	3925
Trimming data between required Dates	44410	3925
Removal of Features with high NaN Values	44410	3854
Removal of Not Required Features*	44410	1945
Removal of outliers and drifted features	44410	1913
Dropping Samples with NaN values	35265	1913

Figure 4 - Low Carbon Caster 2 Data Analysis and Data Filtering Result

* Few features that don't contribute to the model in general were removed. Feature classes to remove were Oscillator Friction Force, Heat Track cut Length and Seq Counter. And certain features are not considered such as last two rows (6 and 7) of Thermocouples, 4 left (A, B, C, D) and 4 right (L, M, N, O) columns of thermocouples at the moment. They will be included in future in modeling.

1.4 Feature Scaling

After removing the Nan values and filtering the features, we have obtained a filtered dataframe. The features present in that data will be further scaled for ML modeling and LIME analysis. The scaling technique used here is Robust Scaler. The **Robust Scaler** is less sensitive to outliers compared to methods like **StandardScaler** or **MinMaxScaler**. It works by removing the median and scaling the data according to the **Interquartile Range (IQR)**. During modeling, only train data was used to create feature scaler and, validation and test data are transformed using the same scalar.

$$X_{scaled} = \frac{X - median(X)}{IQR(X)}$$

where:

- **median(X)** = 50th percentile
- **IQR(X)** = 75th percentile – 25th percentile

2 Data Imbalance

Following table shows the data points with rejection rate for different combinations of casters, steel group and model level. For low carbon and for slab wise model, rejection rate of train, validation and test data is between 2.5 to 4 %. While, for the medium carbon and slab wise model, rejection rate changes drastically on test data (4.7%) as compared train (15%) and validation data (15%).

Caster	Steel Group	Model Level	Train			Validation			Test		
			Accepted	Rejected	% Rejection	Accepted	Rejected	% Rejection	Accepted	Rejected	% Rejection
1	LC (SAE1006)	Slab	21174	636	3.00	4296	187	4.37	7695	200	2.6
		Heat	4867	795	16.33	954	158	16.56	1339	1558	11.58
1	MC (SS400, S275JR)	Slab	3915	604	15.43	333	50	15.02	1174	56	4.77
		Heat	796	383	48.12	108	38	35.19	199	38	19.1
2	LC (SAE1006)	Slab	22417	596	2.66	5311	140	2.64	6626	175	2.64
		Heat	5050	582	11.52	965	128	13.26	1319	145	10.99
2	MC (SS400, S275JR)	Slab	4950	551	11.13	714	37	5.18	687	17	2.47
		Heat	899	327	36.37	127	21	16.54	156	14	8.97
1 & 2	LC (SAE1006)	Slab	43591	1232	2.83	9607	327	3.4	14321	375	2.62
		Heat	9917	1377	13.88	1919	286	14.90	2658	300	11.29
1 & 2	MC (SS400, S275JR)	Slab	8865	1155	13.03	1047	87	8.31	1861	73	3.92
		Heat	1695	710	41.88	235	59	25.11	355	52	14.64

Figure 5 shows the total number of samples with rejection rate (%) for different casters (1, 2, 1&2), steel grad (LC – Low carbon, MC- Medium Carbon) and model level (slab level and heat level).

3 Modelling Technique

A preliminary evaluation using PyCaret compared the performance of different machine-learning models. **XGBoost consistently outperformed other algorithms**, making it the preferred choice for developing the supervised ML framework.

Table 2 - Pycaret Preliminary Evaluation of various ML Models

Model	AUC_ROC	AUC_PR
Extreme Gradient Boosting	0.79	0.022
Light Gradient Boosting Machine	0.79	0.017
Naive Bayes	0.67	0.424
Linear Discriminant Analysis	0.77	0.041
Extra Trees Classifier	0.78	0.029
Gradient Boosting Classifier	0.78	0.017
AdaBoost Classifier	0.77	0.018
Random Forest	0.74	0.052
K-Nearest Neighbors	0.66	0.309
Quadratic Discriminant Analysis	0.50	0.515
Dummy Classifier	0.50	0.515

3.1 Hyperparameter Grid

The XGBoost hyperparameters considered for tuning included the following:

Table 3 - Hyperparameter ranges used for parameter tuning

Parameter	Values
n_estimators	30, 500
max_depth	3, 15
gamma	0, 0.8
scale_pos_weight	0.1, 50
random_state	42
learning_rate	0.05, 0.3
min_child_weight	7, 15
subsample	0.5, 0.8
colsample_bytree	0.6, 0.9

3.2 Experimental Meta-Parameter Grid

One of the key metrics used for ML modeling is β -F1 Score. This metric was further modified to give importance to precision or recall.

$$F_{\beta} = (1 + \beta^2) * \left(\frac{\text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}} \right)$$

The variable used here is called Beta (β).

If $\beta > 1 \rightarrow$ Recall is more weightage.

If $\beta < 1 \rightarrow$ Precision is more weightage.

For each experiment, a meta-parameter grid was constructed. The grid consisted of:

- **β values** for calculation of β -F1
- **Weight combinations** between AUC and β -F1 and used for optimization
 - The optimization criterion was defined as:

$$\text{Optimization} = w \times (\beta\text{-F1_val}) + (1 - w) \times \text{AUC_val}$$

Where: $w \in \{0.1, 0.25, 0.5, 0.75, 1\}$ and $\beta \in \{1, 1.5, 1.75, 2, 2.5, 3\}$

Each experiment was executed using a fixed pair of (β , w) values and each experiment was having 100 optuna trials.

3.3 Optuna

Optuna is an automated hyperparameter optimization framework designed for both single-objective and multi-objective optimization. It primarily works based on **Bayesian optimization techniques** to find the suitable hyperparameter configuration.

Key Components:

1. Objective Function:

For fixed (β, w) , function for validation dataset is calculated which needs to be maximized by varying model hyper-parameters. For this model,

$$\text{Function } f(\beta\text{-f1, auc}) = w \times (\beta\text{-f1}) + (1 - w) \times (\text{auc})$$

objective function : maximize $f(\text{f1, auc})$

2. Study and Trials:

In Optuna, a study is to optimize the given objective function which consists of multiple trials. Each *Trial* corresponds to one evaluation of the model using a specific hyperparameter combination.

Each study was run for **100 trials** with fixed meta-parameters (β, w) with total combinations of 30 (5x6) studies.

The process was automated using **Papermill in Jupyter notebook**, enabling execution with varying meta-parameter inputs. Inputs to each experiment included:

1. β value for calculating $\beta\text{-f1}$
2. weight (w) value for calculating objective function $(w \times (\beta\text{-f1}) + (1 - w) \times (\text{auc}))$
3. $n_trials = 100$

After each trial, all computed metrics were stored in a structured dataframe. The list of metrics are F1, $\beta\text{-F1}$ score, ROC-AUC, PR-AUC (Precision-Recall Area Under Curve), Accuracy, Precision, Recall.

3.4 Best Hyperparameter

After completing 100 trials, Optuna returns the best hyper-parameters which corresponds to the maximum value of the defined objective function.

3.5 Selecting the Best Model

3.5.1 Current Approach - Multi-Objective Optimization

We evaluate experiments using β values [0.5, 0.85, 1, 1.25, 1.5] and weight values [01, 1.5, 1.75, 2, 2.5, 3], leading to **thirty total studies**. For each study, the best model (from Optuna) and its metrics were recorded as shown in Figure 6.

	auc	pr_auc	tn	fp	fn	tp	recall	precision	f1	beta_f1	experiment_	w_opt_func	beta	dataset
1	0.750721	0.169936	9255	352	241	86	0.262997	0.196347	0.224837	0.224837	1	0.1	1	val
4	0.733877	0.169552	9264	343	230	97	0.296636	0.220455	0.252934	0.268127	10	1	1.5	val
7	0.747388	0.155266	9198	409	233	94	0.287462	0.186879	0.226506	0.253832	11	0.1	1.75	val
10	0.740020	0.148055	9142	465	231	96	0.293578	0.171123	0.216216	0.249610	12	0.25	1.75	val
13	0.753063	0.171812	9319	288	242	85	0.259939	0.227882	0.242857	0.251239	13	0.5	1.75	val
16	0.731587	0.152961	9157	450	232	95	0.290520	0.174312	0.217890	0.249566	14	0.75	1.75	val
19	0.744167	0.166208	9229	378	235	92	0.281346	0.195745	0.230866	0.254003	15	1	1.75	val
22	0.751961	0.166409	9260	347	239	88	0.269113	0.202299	0.230971	0.252438	16	0.1	2	val
25	0.739566	0.163056	9185	422	230	97	0.296636	0.186898	0.229314	0.265463	17	0.25	2	val
28	0.740767	0.149280	9219	388	236	91	0.278287	0.189979	0.225806	0.254617	18	0.5	2	val
31	0.735962	0.151916	9176	431	232	95	0.290520	0.180608	0.222743	0.258997	19	0.75	2	val
34	0.751279	0.164202	9326	281	247	80	0.244648	0.221607	0.232558	0.232558	2	0.25	1	val
37	0.709921	0.123134	8833	774	214	113	0.345566	0.127396	0.186161	0.257403	20	1	2	val
40	0.752631	0.177170	9378	229	250	77	0.235474	0.251634	0.243286	0.237578	21	0.1	2.5	val
43	0.744865	0.165215	9217	390	232	95	0.290520	0.195876	0.233990	0.272368	22	0.25	2.5	val
46	0.737858	0.168921	9200	407	228	99	0.302752	0.195652	0.237695	0.281498	23	0.5	2.5	val
49	0.736359	0.151616	9108	499	227	100	0.305810	0.166945	0.215983	0.274335	24	0.75	2.5	val
52	0.730142	0.124355	8883	724	217	110	0.336391	0.131894	0.189492	0.277126	25	1	2.5	val
55	0.752740	0.168583	9263	344	240	87	0.266055	0.201856	0.229551	0.257854	26	0.1	3	val
58	0.746171	0.161762	9089	518	226	101	0.308869	0.163166	0.213531	0.283549	27	0.25	3	val
61	0.744292	0.171670	9122	485	225	102	0.311927	0.173765	0.223195	0.288952	28	0.5	3	val
64	0.707087	0.105177	8767	840	217	110	0.336391	0.115789	0.172279	0.282558	29	0.75	3	val
67	0.745238	0.178873	9415	192	254	73	0.223242	0.275472	0.246622	0.246622	3	0.5	1	val
70	0.723870	0.141793	8965	642	218	109	0.333333	0.145140	0.202226	0.295073	30	1	3	val
73	0.749714	0.177438	9285	322	232	95	0.290520	0.227818	0.255376	0.255376	4	0.75	1	val
76	0.724885	0.165003	9480	127	262	65	0.198777	0.338542	0.250482	0.250482	5	1	1	val
79	0.750935	0.166475	9368	239	250	77	0.235474	0.243671	0.239502	0.237937	6	0.1	1.5	val
82	0.744318	0.164824	9125	482	228	99	0.302752	0.170396	0.218062	0.244352	7	0.25	1.5	val
85	0.742392	0.161742	9152	455	227	100	0.305810	0.180180	0.226757	0.251792	8	0.5	1.5	val
88	0.740579	0.150732	9245	362	239	88	0.269113	0.195556	0.226512	0.241198	9	0.75	1.5	val

Figure 6 Metrics for validation dataset for 30 experiments

After aggregating results:

1. **Pareto Frontier analysis** was performed for AUC and F1 metrics of validation dataset
2. The optimal studies using pareto frontier technique were shortlisted as shown in Figure 7.

	auc	pr_auc	tn	fp	fn	tp	recall	precision	f1	beta_f1	experiment_	w_opt_func	beta	dataset
13	0.753063	0.171812	9319	288	242	85	0.259939	0.227882	0.242857	0.251239	13	0.50	1.75	val
61	0.744292	0.171670	9122	485	225	102	0.311927	0.173765	0.223195	0.288952	28	0.50	3.00	val
55	0.752740	0.168583	9263	344	240	87	0.266055	0.201856	0.229551	0.257854	26	0.10	3.00	val
58	0.746171	0.161762	9089	518	226	101	0.308869	0.163166	0.213531	0.283549	27	0.25	3.00	val
70	0.723870	0.141793	8965	642	218	109	0.333333	0.145140	0.202226	0.295073	30	1.00	3.00	val

Figure 7 Metrics for validation after applying pareto frontier technique

3. Among these, the final selection was made by choosing the model with the highest PR-AUC which is study (experiment) number 13 (Figure 7) and corresponding metrics for train, validation and test are shown in Figure 8.

	auc	pr_auc	tn	fp	fn	tp	recall	precision	f1	beta_f1	experiment	w_opt_func	beta	dataset
12	1.000000	1.000000	43193	398	0	1232	1.000000	0.755828	0.860936	0.926337	13	0.5	1.75	train
13	0.753063	0.171812	9319	288	242	85	0.259939	0.227882	0.242857	0.251239	13	0.5	1.75	val
14	0.780675	0.161951	14103	218	307	68	0.181333	0.237762	0.205749	0.192584	13	0.5	1.75	test

Figure 8 Metrics for best experiment (13) for train , validation and test dataset

The results for all the casters and steel grades for slab-wise model is shown in Figure 9. Maximum AUC on the test data for low carbon is around 0.78 (Figure 9 (a)) for combined caster-1 & 2. While maximum F1 was around 0.21 (Figure 9 (b)) for combined caster-1 & 2.

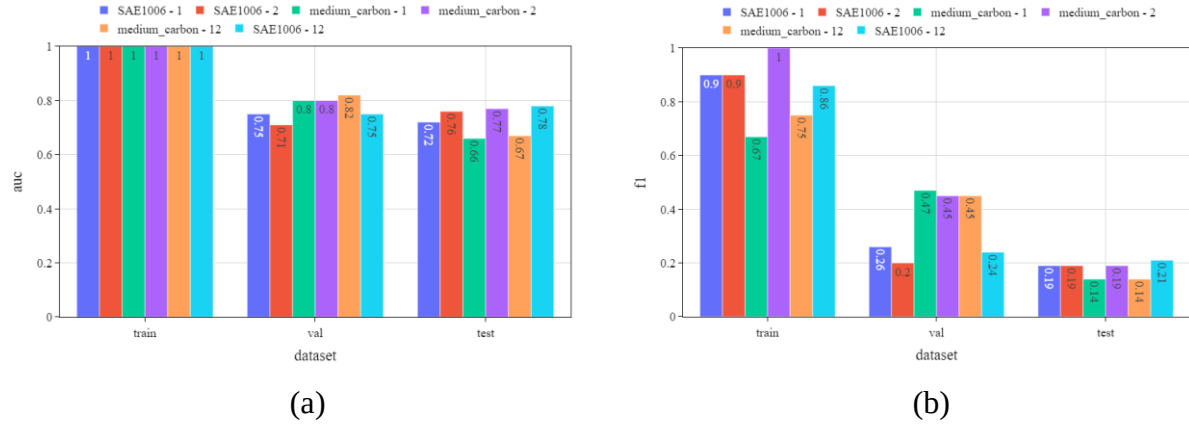


Figure 9 (a) shows the AUC score achieved by low carbon and medium carbon with different caster data and (b) shows the F1 Score achieved for those respective models.

3.5.2 Approach in-Progress (Constraint based optimization in Optuna)

In the current approach (multi-objective optimization), several models are trained with different meta parameters (β for β -F1 and w for objective function). After that, pareto frontier technique is used to filter best model and then pr_AUC was used to select the best model. Instead of the current approach, constraint-based optimization was explored in the Optuna. In Optuna, user can specify the set of constraints, for example, $AUC > 0.7$. Optuna finds different hyper parameters and compute corresponding AUC value for validation data. If AUC is more than 0.7 then Optuna consider it as feasible trial and provide more weightage to a specific hyper-parameter configuration for finding another hyper-parameters configuration. If the AUC is less than 0.7, then Optuna consider it as infeasible trial and provide less weightage to a specific hyper-parameter configuration to find the next hyper-parameters configuration.

This approach was combined with timeseries K-fold cross validation for combined caster – 1 and caster - 2 data. In this approach, train and validation data is combined for K-fold validation. Split in each fold is mentioned in Table 4.

Table 4 Timeseries K-fold splits for 5 folds

Fold	Train Index	Validation Index
1	0 - 9126	9127 - 18252
2	0 - 18252	18253 - 27378
3	0 - 27378	27379 - 36504
4	0 - 36504	36505 - 45630
5	0 - 45630	45631 - 54756

The constraints used for selecting the best hyperparameters are $AUC > 0.7$, $PR_AUC > 0.1$, and $recall > 0.3$. These constraint values are chosen based on prior experience. Within these boundaries, the objective in Optuna is to maximize precision in order to identify the optimal hyperparameters. For each trial, the average AUC, PR_AUC, recall, and precision are computed across five folds.

During hyperparameter tuning, the predicted probabilities for the validation set from all folds and all trials are stored. After determining the best hyperparameters, the model is retrained using the complete dataset (training + validation). The probability threshold is then derived using the stored validation probabilities from the best trial. The threshold is selected such that recall remains above 0.3 while precision is maximized. This threshold is subsequently used to evaluate the test dataset, and the resulting metrics are summarized in Table 5.

This approach improves overall performance and removes the need to build multiple models or perform a separate model selection process.

Table 5 Comparison of the metrics for different approaches

approach	auc	pr_auc	f1	tn	fp	fn	tp	beta_f1	precision	recall	best_th	dataset
Multi-objective Optimization	0.77	0.15	0.22	13838	483	271	104	0.25	0.18	0.28	0.04	test

Constraint Based optimization	0.81	0.18	0.23	13854	467	266	109	0.24	0.19	0.29	0.11	test
-------------------------------	------	------	------	-------	-----	-----	-----	------	------	------	------	------

The current approach is applied only to low-carbon data and caster 1 and 2, as the large volume of data available for these categories makes them well-suited for K-fold validation.

4 LIME Analysis

LIME (Local Interpretable Model-Agnostic Explanations) is an Explainable AI method used to interpret predictions from complex black-box models by approximating them with simple, interpretable models in the local neighborhood of each prediction.

During the LIME analysis:

- The parameter ranges that appeared most frequently in validation among predicted slabs as rejected were identified.
- Additionally, features previously known to contribute to the root cause were selected to form the initial set of constraints.
- Building on these initial constraints, further constraints were introduced to improve overall precision.

Table 6 The table shows the query and corresponding number of slabs and rejection rate from validation data.

Query	Slab count in validation	Rejection rate (%)
pred == 1 and DMS_WS_POS_mean > 23.80748	37	25.8
pred == 1 and DMS_WS_POS_mean > 23.80748 and TP_FIX_GH_grad_min > 15.6306	14	28.5
pred == 1 and DMS_WS_POS_mean > 23.80748 and TP_FIX_GH_grad_min > 15.6306 and TP_LSE_H2_std > 2.7664	11	32.35
pred == 1 and DMS_WS_POS_mean > 23.80748 and TP_LSE_H2_std > 2.76646 and TP_FIX_GH_grad_min > 15.6306 and TP_FIX_HI_grad_min > 13.58434	9	34.61

The sensor data corresponding to the above constraint for the specific slab is shown in the figures below. In Figure 10, the vertical red lines shows the entry and exit points of the defective slab. The parameter DMS_WS_POS (stopper-rod position) shows a continuous upward trend, and when its absolute value exceeds a certain threshold (23.8, as specified in the constraint). Along with the increasing stopper-rod position, the Euclidean distances (TP_FIX_HI_grad_mean) for **column H** (3 rows) and **column I** (3 rows) also increase locally, which may be attributed to a drop in the temperature of column H, as illustrated in Figure 10. A similar temperature-related behavior is observed for thermocouples in **columns G and H** in **Figure 9**. Additionally, it is observed that the casting speed remains steady at **4.6 m/min** in the region highlighted by the red lines in Figure 11. This combined effect can be associated with the clogging, process phenomena in casting leading to defect. The query and quality detail of the coil highlighted between redline is mentioned below,

Query : pred == 1 and DMS_WS_POS_mean > 23.80748 and TP_LSE_H2_std > 2.76646
and TP_FIX_GH_grad_min > 15.6306 and TP_FIX_HI_grad_min > 13.58434

Coil ID: 72240508804

Quality of the coil : Rejected by Customer

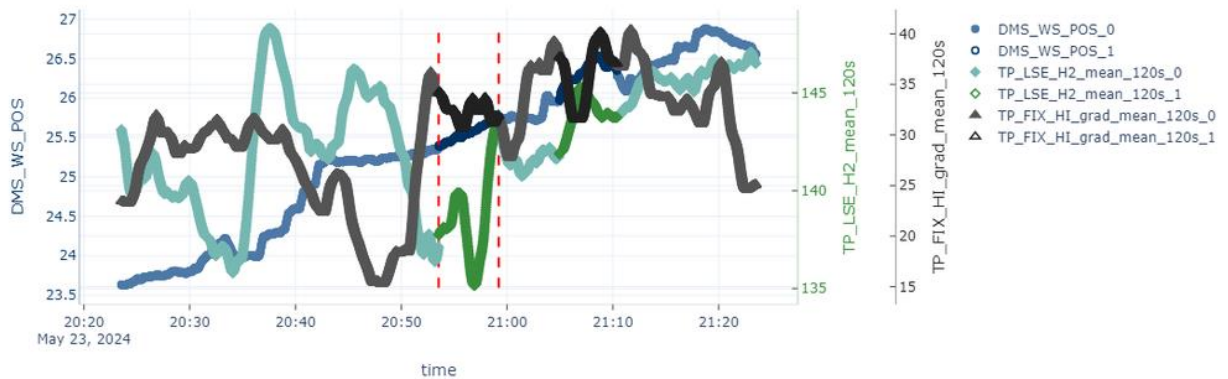


Figure 10 shows the DMS_WS_POS with temperature gradient (TP_FIX_HI_grad) and thermocouple temperature of H2

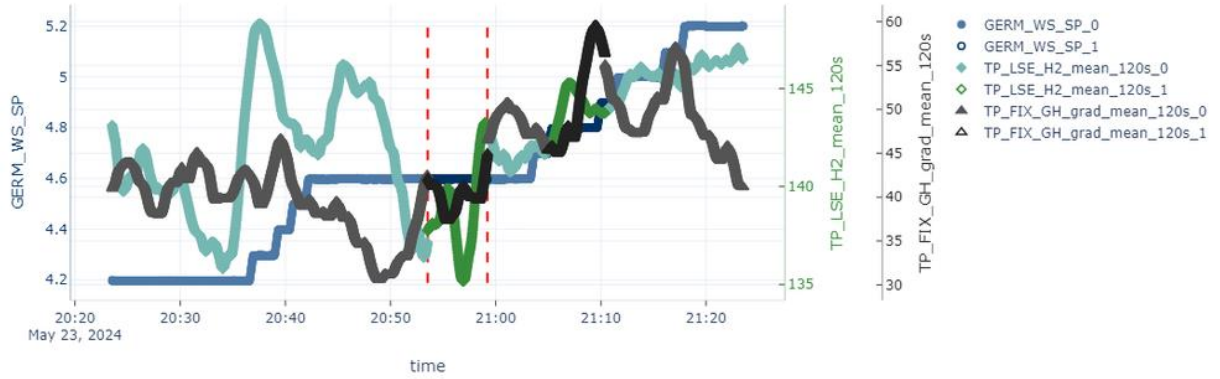


Figure 11 - shows the casting speed (GERM_WS_SP) with temperature gradient (TP_FIX_GH_grad) and thermocouple temperature of H2

Similarly, additional constraints were identified and linked to specific root causes based on prior discussions with the technical expert. These LIME-derived constraints enable roughly 34% of defective slabs to be correctly detected (representing recall) through interpretable, feature-based constraints, with an accompanying precision of about 30%. The distribution of the corresponding root causes is shown in Figure 12

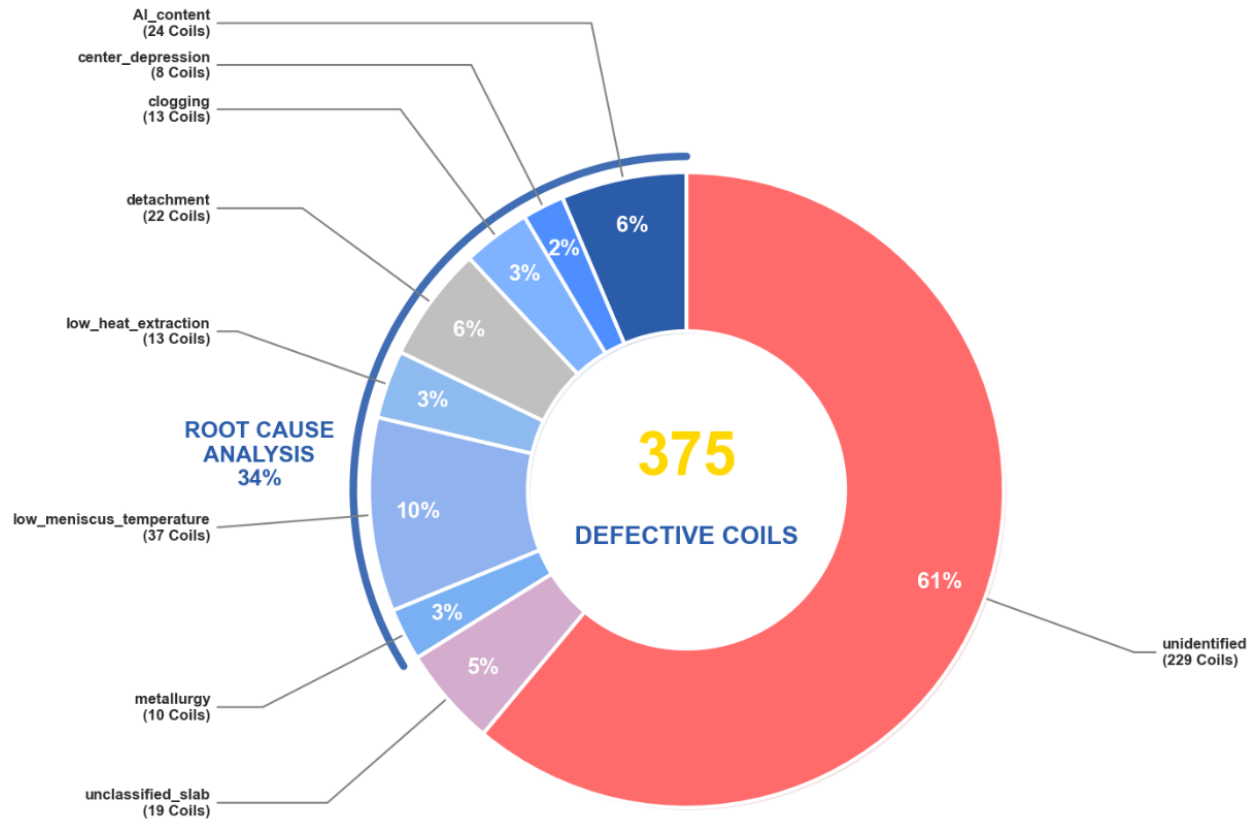


Figure 12 - Defect Prediction pie-chart based on identified constraints

5 Challenges

5.1 Selecting metric for optimization

As discussed, the primary challenge with multi-objective optimization is selecting the best model from the many models generated using different weight combinations in the multi-objective function. Choosing an appropriate objective function itself is often difficult, as it can be formulated using various metrics such as β -F1, AUC, PR_AUC, precision, recall, and others. Additionally, determining the correct weightage for these metrics typically requires a Design of Experiments (DOE) approach or must be guided by specific business/technical requirements. Because of this complexity, there is no systematic way to deploy this method for automated retraining at the plant/site level.

To overcome these limitations, a constraint-based optimization approach was investigated. This method appears more suitable, as it produces a single model with the best hyperparameters and is

simpler to deploy. However, it has not yet been extensively tested. Using this approach, all configurations will be executed, and their performance compared against the multi-objective optimization method.

5.2 Improving the prediction

The model's performance requires improvement, particularly with respect to precision and recall. At present, the recall and precision for low-carbon slabs on casters 1 and 2 are 0.29 and 0.19, respectively, based on the model's test dataset. To enhance these metrics, more advanced feature engineering and feature selection techniques will be applied to strengthen the predictive capability of the model.

5.3 Classifying the positive predicted coils in root cause classes

The primary objective of this project is to enable real-time predictive actions to prevent defects. To take appropriate corrective measures, it is essential to identify the root causes associated with coils predicted as positive. Using LIME, the model provides several ranges of feature values that contribute to classifying these coils as positive. However, automatically filtering the correct feature ranges and mapping them to specific root causes is challenging, and solving this remains a key problem. Several brainstormed ideas to address this challenge are outlined below:

1. K-means Clustering of the ranges for positive predicted samples:

The idea is to cluster the samples based on the feature ranges identified by LIME. Ideally, each cluster should correspond to one or more potential root causes, which can then be further refined into a specific root cause using process knowledge. In other words, if a particular cluster is associated with predefined root cause with ranges of certain features, the corresponding cluster can be mapped to that specific root cause.

2. Creating a root cause specific model based on general model

Based on the root-cause insights obtained from the general model, separate root-cause-specific models can be trained using parameters relevant to each specific cause. When such a model produces a positive prediction, it can be directly associated with that root cause and corresponding action can be taken appropriately.

6 Current Deep Learning Approach (Under Progress)

Deep Learning approach is used to modelling at defect-level.

To start with we considered the following categories for modelling the preliminary deep learning models

- Model Level: Defect Level
- Steel Type: Low Carbon (SAE1006)
- Caster: TSC1
- Defects side: Top
- Model Type: LSTM and TCN
- Time period for Modelling: 01/08/2023 to 14/07/2024

6.1 Input and targets

Inputs: Unlike in the machine learning models (where we used features), raw parameter values from sensors (resampled to 1-second intervals) were directly used for modeling. The total sensor parameters/tag used are 165. The same data filtering and preprocessing methods applied in the ML modeling were employed, including removal of NaN values, exclusion of out-of-range values, and similar scaling techniques.

Targets; For the target variable, we focused exclusively on top-side defects (longitudinal cracks (LC)) detected by the IMS system at 1-second resolution. This choice was made due to a high rate of false positives on the bottom side of the coil, may be caused by poor lighting conditions and other technical factors.

Note: Defects identified spatially along the coil are traced back to caster mold to get corresponding temporal defect information.

Data Split:

- Train samples = 7233523 (defective rate: 2.249%)
- Validation samples = 1736055 (defective rate: 2.760%)
- Test samples = 2281492 (defective rate: 1.829%)

6.2 Modelling

We are considering sequence models such as Long short Term Memory LSTM and Temporal Convolutional Networks TCN models as our preliminary model to start with for our analysis.

6.2.1 Hyperparameters tuning

The hyperparameters considered for tuning the LSTM model are following:

Table - Hyperparameter ranges used for parameter tuning

Parameter	Values
Sequence length (SL)	30, 60, 120 240
Sequence stride window	(SL)/2, (SL/3), (SL/5)
Batch size	32, 64, 128
Hidden size	32, 64, 128
Number of Layers	1, 2, 3, 5
Dropout rate	0.3, 0.5
Learning rate	10e-2, 10e-3, 10e-4,
Layer normalization	True, False
Loss Type	BCEwithLogits
Optimizer	AdamW
Early stop (patience)	20
ReduceLROnPlateau	Patience = 10, factor= 0.5
Device	GPU

Grid search is used to for finding optimum hyperparameters.

6.2.2 Metrics for model performance evaluation

Again, to start with and to understand whether if model is learning something we used ROC-AUC to evaluate the performance of the model.

6.2.3 Chunk wise training, time series sequences and corresponding targets generation

Due to limited computational resources for handling large-scale time-series data, we adopt a chunk-wise training strategy. First, the complete dataset is split into training, validation, and test sets. Each split is then further divided into smaller temporal chunks, where each chunk corresponds to 31 consecutive days of data.

During model training, chunks from the training split are loaded one at a time. For each chunk, time-series sequences and their corresponding targets are generated on the fly using a predefined window size, stride, and target-generation mode (`y_mode`). After the model has processed all chunks in the training split, one full epoch of training is completed. The same procedure is applied during validation, and finally during testing.

From each chunk, fixed-length time-series input sequences are constructed from the input features. For each sequence, a corresponding target variable is generated based on different labeling strategies. We define multiple target-generation modes (`y_mode`) to capture different variations:

1. ***last***: The target corresponds to the value at the last timestep of the input window.
2. ***next***: The target is the value at the timestep immediately following the input window, enabling next-step forecasting.
3. ***Mean_thresh***: The target is set to 1 if mean value over the window is greater than certain threshold we provide based on our technical reasoning
4. ***seq***: The target is the entire output sequence corresponding to the input window, enabling many-to-many sequence learning.

```
class LSTMClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, use_ln=False, dropout=0.0):
        super().__init__()
        self.hidden_size = hidden_size
        self.lstm = nn.LSTM(
            input_size, hidden_size, num_layers, batch_first=True,
            dropout=(dropout if num_layers > 1 else 0.0)
        )
        self.post_drop = nn.Dropout(dropout) if dropout > 0 else nn.Identity()
        self.ln = nn.LayerNorm(hidden_size) if use_ln else None
        self.fc = nn.Linear(hidden_size, 1)
        self.init_weights()
    def init_weights(self):
        for name, p in self.lstm.named_parameters():
            if 'weight_ih' in name: nn.init.xavier_uniform_(p)
            elif 'weight_hh' in name: nn.init.orthogonal_(p)
            elif 'bias' in name:
                p.data.fill_(0)
                p.data[self.hidden_size:2*self.hidden_size].fill_(1.0)
        nn.init.xavier_uniform_(self.fc.weight); self.fc.bias.data.fill_(0)
    def forward(self, x):
        _, (hn, _) = self.lstm(x)
        h = hn[-1]
        if self.ln is not None: h = self.ln(h)
        h = self.post_drop(h)
        return self.fc(h).squeeze(1)
```

Code snippet of LSTM class